BAKERY WHOLESALE DISTRIBUTION ROUTE DATABASE

Database Management System Case Study

Case Study 35 | Implemented in PostgreSQL

Student Name	Jaikishan Suthar
Roll Number	150096752152
Course	B.Tech CSE
Subject	Database Management System (DBMS)

Declaration

I hereby declare that this report titled **"Bakery Wholesale Distribution Route Database"** is the record of the work carried out by me for the Database Management System case study. The database design, SQL scripts, test results and analysis presented in this report were prepared by me for academic purposes.

Jaikishan Suthar
Roll Number: 150096752152
B.Tech CSE

Acknowledgement

I would like to thank my faculty and my institution for the guidance and the opportunity to work on this case study. The project helped me understand relational database design, constraints, SQL analysis, and role-based access control in a practical way.

Table of Contents

1. Introduction 3

2. Problem Statement 3

3. Objectives 3

4. System Requirements 3

5. Database Design 5

6. ER Diagram 6

7. Relational Schema 7

8. Table Descriptions 8

9. Database Constraints 10

10. Referential Integrity 10

11. SQL Implementation 11

12. Sample Data 12

13. SQL Queries and Results 14

14. SQL Concepts Demonstrated 19

15. Roles and Permissions 19

16. Testing 20

17. Results and Discussion 21

18. Advantages 21

19. Limitations 21

20. Future Scope 21

21. Conclusion 22

22. References 22

1. Introduction

The Bakery Wholesale Distribution Route Database is a relational database system developed to manage the distribution of bakery products from a wholesale bakery to multiple retail outlets. The system maintains information about routes, outlets, products, deliveries, returned products, and invoices.

The database is implemented using PostgreSQL and provides structured storage, data integrity, and SQL-based analysis of distribution operations. Each outlet belongs to a delivery route that is served on a fixed weekday; every delivery lists the products supplied, unsold or damaged goods come back as returns, and each outlet is billed through invoices whose payment status is tracked.

The project answers five key business questions with SQL: how much each outlet is actually supplied after returns (net supply), what the return rate of each product is, how many deliveries each route makes per day, which invoices are unpaid, and which outlets return more than 20% of what they receive.

2. Problem Statement

Managing bakery wholesale distribution using manual records can make it difficult to track deliveries, products, returns, routes, and invoices. Duplicate records, incorrect quantities, missing information, and difficulty in generating reports can affect the efficiency of the distribution process.

Bakery products are perishable (shelf life in this database ranges from 2 to 20 days), so quantities that are over-supplied come back as waste. The bakery therefore needs to know exactly what each outlet kept versus returned. Therefore, a centralized relational database is required to store and manage distribution information in a structured and reliable manner.

3. Objectives

- Manage bakery products.
- Manage retail outlets.
- Manage delivery routes.
- Record product deliveries.
- Record returned products.
- Calculate net supply.
- Calculate product return rates.
- Track unpaid invoices.
- Maintain referential integrity.
- Implement database constraints.
- Implement database roles and permissions.
- Generate analytical reports using SQL.

4. System Requirements

Hardware

Component	Requirement
Processor	Intel / AMD processor
RAM	4 GB or above
Storage	500 MB or above

Software

Component	Requirement
Operating System	Windows / macOS / Linux
Database	PostgreSQL
Database Tool	pgAdmin 4
Language	SQL
Diagram Tool	draw.io / diagrams.net

The scripts in this report were also executed and verified on PostgreSQL 16.

5. Database Design

The database contains eight tables. The database name is **new_bakery_distribution**.

No.	Table	Purpose
1	Routes	Delivery routes with a fixed weekday and starting area
2	Outlets	Retail shops served by the bakery; each belongs to one route
3	Products	Bakery items with category, price and shelf life
4	Deliveries	One delivery visit to one outlet on one date
5	Delivery Items	Products and quantities included in each delivery
6	Returns	One return event against a delivery, with a reason
7	Return Items	Products and quantities returned in each return
8	Invoices	Billing and payment status for each outlet

Why the tables are separated

The design follows normalisation. A delivery can contain many products and a product appears in many deliveries, which is a many-to-many relationship. It is resolved with the **Delivery Items** table instead of storing repeating product columns inside Deliveries. The same idea is used for returns: **Returns** stores the return event (date and reason) once, and **Return Items** stores the quantity for each product.

Keeping routes, outlets and products in their own tables means that a route name, an outlet address or a product price is stored only once and changed in one place, which removes duplication and update anomalies. Separating returns from delivery lines also allows the reason and date of each return to be recorded, which a single "returned quantity" column could not do.

6. ER Diagram

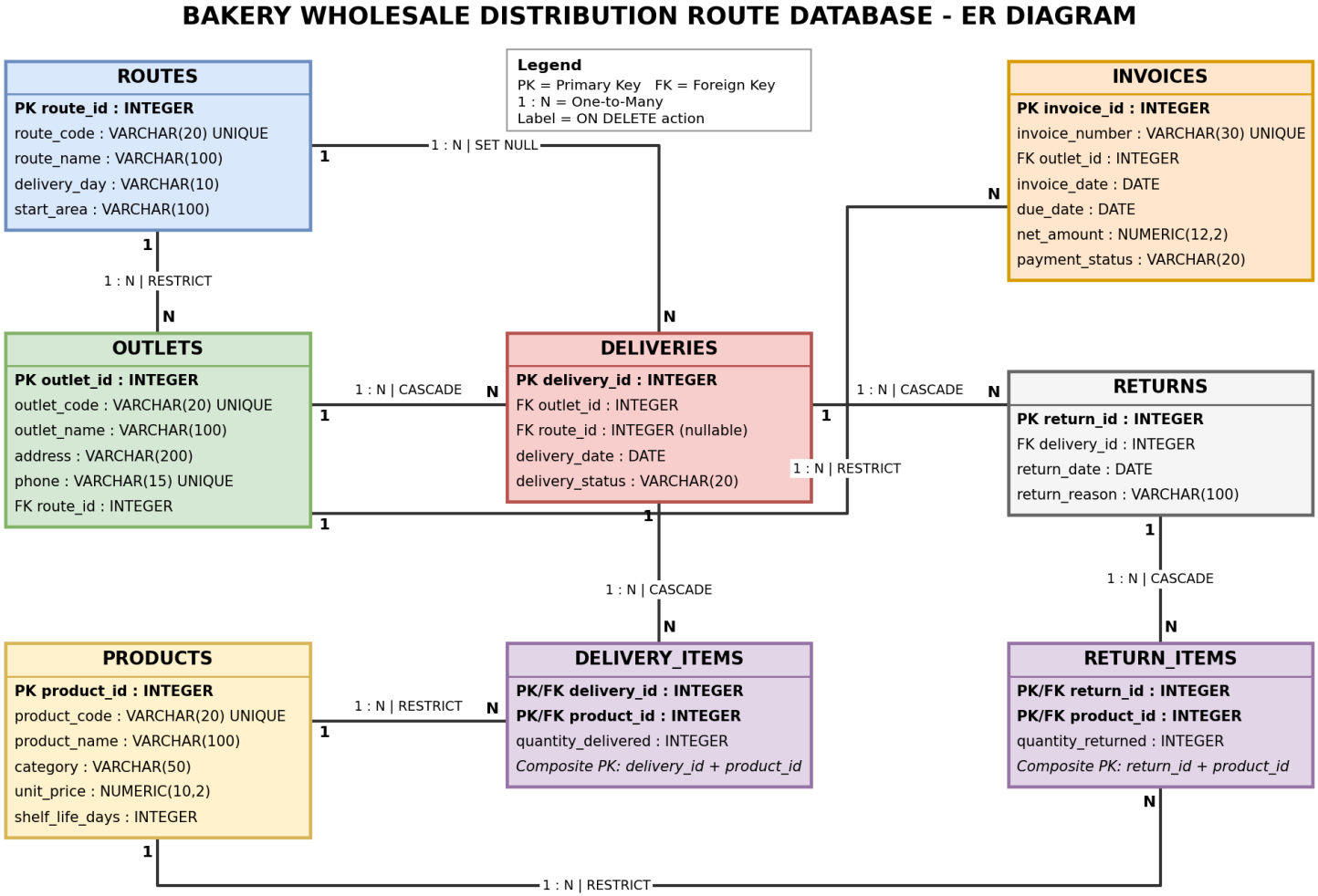


Figure 1: Entity Relationship Diagram (redrawn from Bakery_Distribution_ER_Diagram.drawio)

The ER diagram represents the relationships between routes, outlets, deliveries, products, returns, and invoices. Primary keys uniquely identify records, while foreign keys establish relationships between related entities. Each connector is labelled with its cardinality (1 : N) and the ON DELETE action defined in schema.sql.

7. Relational Schema

Notation: PK = primary key, FK = foreign key, UQ = unique, CK = check constraint.

ROUTES

```
route_id  PK
route_code  UQ
route_name
delivery_day  CK
start_area
```

OUTLETS

```
outlet_id  PK
outlet_code  UQ
outlet_name
address
phone  UQ
route_id  FK -> routes
```

PRODUCTS

```
product_id  PK
product_code  UQ
product_name
category
unit_price  CK (> 0)
shelf_life_days  CK (> 0)
```

DELIVERIES

```
delivery_id  PK
outlet_id  FK -> outlets
route_id  FK -> routes (nullable)
delivery_date
delivery_status  CK, DEFAULT
```

DELIVERY_ITEMS

```
(delivery_id, product_id)  composite PK
delivery_id  FK -> deliveries
product_id  FK -> products
quantity_delivered  CK (> 0)
```

RETURNS

```
return_id  PK
delivery_id  FK -> deliveries
return_date  DEFAULT CURRENT_DATE
return_reason
```

RETURN_ITEMS

```
(return_id, product_id)  composite PK
return_id  FK -> returns
product_id  FK -> products
quantity_returned  CK (> 0)
```

INVOICES

```
invoice_id  PK
invoice_number  UQ
outlet_id  FK -> outlets
invoice_date  DEFAULT
due_date  CK (>= invoice_date)
net_amount  CK (>= 0)
payment_status  CK, DEFAULT
```

8. Table Descriptions

8.1 Routes

Column	Data Type	Constraint	Description
route_id	INTEGER	PK, IDENTITY	Unique route ID
route_code	VARCHAR(20)	UNIQUE, NOT NULL	Route code (e.g. R001)
route_name	VARCHAR(100)	NOT NULL	Route name
delivery_day	VARCHAR(10)	NOT NULL, CHECK	Weekday (Monday to Saturday)
start_area	VARCHAR(100)	NOT NULL	Starting area of the route

8.2 Outlets

Column	Data Type	Constraint	Description
outlet_id	INTEGER	PK, IDENTITY	Unique outlet ID
outlet_code	VARCHAR(20)	UNIQUE, NOT NULL	Outlet code
outlet_name	VARCHAR(100)	NOT NULL	Outlet name
address	VARCHAR(200)	NOT NULL	Outlet address
phone	VARCHAR(15)	UNIQUE	Contact number
route_id	INTEGER	FK, NOT NULL	Route serving the outlet (ON DELETE RESTRICT)

8.3 Products

Column	Data Type	Constraint	Description
product_id	INTEGER	PK, IDENTITY	Unique product ID
product_code	VARCHAR(20)	UNIQUE, NOT NULL	Product code
product_name	VARCHAR(100)	NOT NULL	Product name
category	VARCHAR(50)	NOT NULL	Bread, Buns, Cake, Cookies...
unit_price	NUMERIC(10,2)	NOT NULL, CHECK > 0	Price per unit
shelf_life_days	INTEGER	NOT NULL, CHECK > 0	Shelf life in days

8.4 Deliveries

Column	Data Type	Constraint	Description
delivery_id	INTEGER	PK, IDENTITY	Unique delivery ID
outlet_id	INTEGER	FK, NOT NULL	Receiving outlet (ON DELETE CASCADE)
route_id	INTEGER	FK, nullable	Route used (ON DELETE SET NULL)
delivery_date	DATE	NOT NULL	Date of delivery
delivery_status	VARCHAR(20)	NOT NULL, DEFAULT, CHECK	Delivered or Cancelled (default Delivered)

8.5 Delivery Items

Column	Data Type	Constraint	Description
delivery_id	INTEGER	PK (part), FK	Delivery (ON DELETE CASCADE)
product_id	INTEGER	PK (part), FK	Product (ON DELETE RESTRICT)
quantity_delivered	INTEGER	NOT NULL, CHECK > 0	Units delivered

8.6 Returns

Column	Data Type	Constraint	Description
return_id	INTEGER	PK, IDENTITY	Unique return ID
delivery_id	INTEGER	FK, NOT NULL	Delivery being returned against (ON DELETE CASCADE)
return_date	DATE	NOT NULL, DEFAULT	Return date (default CURRENT_DATE)
return_reason	VARCHAR(100)	-	Reason for return

8.7 Return Items

Column	Data Type	Constraint	Description
return_id	INTEGER	PK (part), FK	Return (ON DELETE CASCADE)
product_id	INTEGER	PK (part), FK	Product (ON DELETE RESTRICT)
quantity_returned	INTEGER	NOT NULL, CHECK > 0	Units returned

8.8 Invoices

Column	Data Type	Constraint	Description
invoice_id	INTEGER	PK, IDENTITY	Unique invoice ID
invoice_number	VARCHAR(30)	UNIQUE, NOT NULL	Invoice number
outlet_id	INTEGER	FK, NOT NULL	Billed outlet (ON DELETE RESTRICT)
invoice_date	DATE	NOT NULL, DEFAULT	Invoice date (default CURRENT_DATE)
due_date	DATE	NOT NULL, CHECK	Payment due date, not before invoice date
net_amount	NUMERIC(12,2)	NOT NULL, CHECK >= 0	Invoice amount
payment_status	VARCHAR(20)	NOT NULL, DEFAULT, CHECK	Paid, Unpaid or Partially Paid (default Unpaid)

9. Database Constraints

Primary Key

A primary key uniquely identifies each row and cannot be NULL. Each master table uses an auto-generated integer key: `route_id`, `outlet_id`, `product_id`, `delivery_id`, `return_id`, `invoice_id`. A surrogate integer is short, fast to join and never changes, while business codes such as `route_code` stay UNIQUE.

Foreign Key

A foreign key makes a column refer to the primary key of another table, so a child row cannot point to a parent that does not exist. Examples: `outlets.route_id`, `deliveries.outlet_id`, `delivery_items.product_id`, `returns.delivery_id`, `invoices.outlet_id`.

Composite Primary Key

Two link tables use a primary key made of two columns:

```
delivery_items  (delivery_id, product_id)
return_items    (return_id, product_id)
```

This guarantees that the same product cannot appear twice on one delivery or one return; the quantity of a product is stored in a single row.

NOT NULL

Required information (names, codes, dates, quantities, amounts) cannot be left empty.

UNIQUE

Prevents duplicates in `route_code`, `outlet_code`, `product_code`, `phone`, `invoice_number`.

CHECK

Validates values before they are stored: `unit_price > 0`, `shelf_life_days > 0`, `quantity_delivered > 0`, `quantity_returned > 0`, `net_amount >= 0`, `due_date >= invoice_date`, delivery_day within Monday to Saturday, delivery_status in (Delivered, Cancelled) and payment_status in (Paid, Unpaid, Partially Paid).

DEFAULT

Supplies a value when none is given: `delivery_status = 'Delivered'`, `payment_status = 'Unpaid'`, `return_date = CURRENT_DATE`, `invoice_date = CURRENT_DATE`.

10. Referential Integrity

Relationship	Action
Routes → Outlets	RESTRICT
Outlets → Deliveries	CASCADE
Routes → Deliveries	SET NULL
Deliveries → Delivery Items	CASCADE
Deliveries → Returns	CASCADE
Returns → Return Items	CASCADE
Products → Delivery Items	RESTRICT

Relationship	Action
Products → Return Items	RESTRICT
Outlets → Invoices	RESTRICT

RESTRICT blocks deletion of a parent row while child rows exist. It protects a route that still has outlets, a product that appears in past deliveries or returns, and an outlet that has been invoiced.

CASCADE automatically deletes the dependent rows when the parent is deleted. Deleting a delivery removes its items and returns, and deleting a return removes its return items, so no orphan rows remain.

SET NULL keeps the child row but clears the reference. If a route is retired, its delivery history stays in the database with route_id set to NULL.

Note: Outlets → Invoices is also RESTRICT in schema.sql and in the ER diagram; it is listed here for completeness.

11. SQL Implementation

The project is delivered as SQL script files, which are run in this order in the pgAdmin 4 Query Tool after creating the database `new_bakery_distribution`:

File	Contents
schema.sql	Creates the 8 tables with primary keys, foreign keys, constraints and referential actions
seed.sql	Inserts the sample data into all tables
queries.sql	Contains the five required case-study queries plus additional analytical queries
roles.sql	Creates the bakery_manager and bakery_clerk roles with GRANT and REVOKE (see Section 15)

Example from schema.sql, showing a composite key and referential actions:

```
CREATE TABLE delivery_items (
    delivery_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity_delivered INTEGER NOT NULL,
    CONSTRAINT pk_delivery_items PRIMARY KEY (delivery_id, product_id),
    CONSTRAINT fk_delivery_item_delivery FOREIGN KEY (delivery_id)
        REFERENCES deliveries(delivery_id) ON DELETE CASCADE,
    CONSTRAINT fk_delivery_item_product FOREIGN KEY (product_id)
        REFERENCES products(product_id) ON DELETE RESTRICT,
    CONSTRAINT chk_delivery_quantity CHECK (quantity_delivered > 0)
);
```

After running schema.sql and seed.sql, the record counts are:

Table	Records
routes	6
products	10
outlets	12
deliveries	18
delivery_items	48
returns	12
return_items	24
invoices	12

12. Sample Data

Representative data is shown below. Total records: 6 routes, 10 products, 12 outlets, 18 deliveries, 48 delivery items, 12 returns, 24 return items and 12 invoices.

Routes

Route Code	Route Name	Day	Start Area
R001	Navi Mumbai North	Monday	Vashi
R002	Navi Mumbai South	Tuesday	Nerul
R003	Mumbai Central	Wednesday	Sion
R004	Thane East	Thursday	Thane
R005	Thane West	Friday	Mulund
R006	Panvel Route	Saturday	Panvel

Products

Code	Product	Category	Unit Price	Shelf Life (days)
P001	White Bread	Bread	40.00	3
P002	Brown Bread	Bread	50.00	4
P003	Burger Buns	Buns	60.00	4
P004	Pav	Buns	35.00	3
P005	Chocolate Cake	Cake	450.00	3
P006	Vanilla Cake	Cake	400.00	3
P007	Butter Cookies	Cookies	120.00	15
P008	Chocolate Cookies	Cookies	140.00	15
P009	Rusk	Bakery	90.00	20
P010	Croissant	Pastry	80.00	2

Outlets

Code	Outlet	Address	Route
O001	Fresh Mart Vashi	Sector 17, Vashi	R001
O002	Daily Needs Vashi	Sector 10, Vashi	R001
O003	Shree Grocery Nerul	Sector 8, Nerul	R002
O004	Sai Supermarket Nerul	Sector 20, Nerul	R002
O005	City Mart Sion	Sion East	R003
O006	Royal Stores Sion	Sion West	R003
O007	Thane Fresh	Naupada, Thane	R004
O008	Om Supermarket	Khopat, Thane	R004
O009	Westside Grocery	Mulund West	R005
O010	Green Basket	Mulund East	R005
O011	Panvel Daily	Old Panvel	R006
O012	New Panvel Mart	New Panvel	R006

Invoices (first rows)

Invoice	Outlet ID	Invoice Date	Due Date	Net Amount	Status
INV-1001	1	2026-09-21	2026-10-05	5200.00	Paid
INV-1002	2	2026-09-21	2026-10-05	4800.00	Unpaid
INV-1003	3	2026-09-22	2026-10-06	6900.00	Paid
INV-1004	4	2026-09-22	2026-10-06	7500.00	Unpaid
INV-1006	6	2026-09-23	2026-10-07	5800.00	Partially Paid

13. SQL Queries and Results

All queries below were executed on the seeded database (PostgreSQL 16); the outputs shown are the actual results.

Query 1 — Net Supply

Purpose: Calculate the quantity actually supplied to each outlet for each product after subtracting returned quantities (Net Supply = Delivered - Returned).

```
SELECT
    o.outlet_name,
    p.product_name,
    SUM(di.quantity_delivered) AS total_delivered,
    COALESCE(SUM(ri.quantity_returned), 0) AS total_returned,
    SUM(di.quantity_delivered)
        - COALESCE(SUM(ri.quantity_returned), 0) AS net_supply
FROM outlets o
JOIN deliveries d
    ON o.outlet_id = d.outlet_id
JOIN delivery_items di
    ON d.delivery_id = di.delivery_id
JOIN products p
    ON di.product_id = p.product_id
LEFT JOIN returns r
    ON d.delivery_id = r.delivery_id
LEFT JOIN return_items ri
    ON r.return_id = ri.return_id
    AND di.product_id = ri.product_id
GROUP BY
    o.outlet_id,
    o.outlet_name,
    p.product_id,
    p.product_name
ORDER BY
    o.outlet_name,
    p.product_name
```

Result:

outlet_name	product_name	total_delivered	total_returned	net_supply
City Mart Sion	Brown Bread	150	8	142
City Mart Sion	Butter Cookies	75	0	75
City Mart Sion	Vanilla Cake	25	5	20
Daily Needs Vashi	Chocolate Cookies	25	0	25
Daily Needs Vashi	Pav	130	8	122
Daily Needs Vashi	White Bread	170	12	158
Fresh Mart Vashi	Burger Buns	100	5	95
Fresh Mart Vashi	Butter Cookies	30	0	30
Fresh Mart Vashi	White Bread	200	10	190
Green Basket	Brown Bread	100	20	80
Green Basket	Chocolate Cookies	35	8	27
Green Basket	Rusk	40	0	40
New Panvel Mart	Brown Bread	100	10	90
New Panvel Mart	Butter Cookies	40	0	40
New Panvel Mart	Pav	80	12	68
Om Supermarket	Brown Bread	90	12	78

outlet_name	product_name	total_delivered	total_returned	net_supply
Om Supermarket	Pav	75	10	65
Om Supermarket	Rusk	30	0	30
Panvel Daily	Burger Buns	70	7	63
Panvel Daily	Chocolate Cake	25	0	25
Panvel Daily	White Bread	130	18	112
Royal Stores Sion	Chocolate Cookies	70	0	70
Royal Stores Sion	Pav	80	10	70
Royal Stores Sion	White Bread	220	25	195
Sai Supermarket Nerul	Burger Buns	60	12	48
Sai Supermarket Nerul	Croissant	60	0	60
Sai Supermarket Nerul	White Bread	230	30	200
Shree Grocery Nerul	Brown Bread	190	10	180
Shree Grocery Nerul	Chocolate Cake	45	3	42
Shree Grocery Nerul	Rusk	40	0	40
Thane Fresh	Burger Buns	70	0	70
Thane Fresh	Chocolate Cake	20	4	16
Thane Fresh	White Bread	110	15	95
Westside Grocery	Butter Cookies	40	5	35
Westside Grocery	Croissant	20	0	20
Westside Grocery	White Bread	95	8	87

The query returns 36 outlet-product combinations. For example, White Bread at Sai Supermarket Nerul had 230 units delivered and 30 returned, giving a net supply of 200. A LEFT JOIN with COALESCE keeps rows with no returns and shows 0 returned instead of NULL.

Query 2 — Return Rate for Each Product

Purpose: Find the percentage of delivered units that came back, per product (Return Rate = Returned / Delivered x 100).

```
SELECT
    p.product_name,
    SUM(di.quantity_delivered) AS total_delivered,
    COALESCE(SUM(ri.quantity_returned), 0) AS total_returned,
    ROUND(
        COALESCE(SUM(ri.quantity_returned), 0) * 100.0
        / NULLIF(SUM(di.quantity_delivered), 0),
        2
    ) AS return_rate_percent
FROM products p
JOIN delivery_items di
    ON p.product_id = di.product_id
JOIN deliveries d
    ON di.delivery_id = d.delivery_id
LEFT JOIN returns r
    ON d.delivery_id = r.delivery_id
LEFT JOIN return_items ri
    ON r.return_id = ri.return_id
    AND p.product_id = ri.product_id
GROUP BY
    p.product_id,
    p.product_name
ORDER BY
    return_rate_percent DESC
```

Result:

product_name	total_delivered	total_returned	return_rate_percent
Vanilla Cake	25	5	20.00
Pav	365	40	10.96
White Bread	1155	118	10.22
Brown Bread	630	60	9.52
Burger Buns	300	24	8.00
Chocolate Cake	90	7	7.78
Chocolate Cookies	130	8	6.15
Butter Cookies	185	5	2.70
Croissant	80	0	0.00
Rusk	110	0	0.00

Vanilla Cake has the highest return rate (20.00%), but only 25 units were delivered, so the figure rests on a small quantity. Pav (10.96%) and White Bread (10.22%) are the highest among high-volume items. Croissant and Rusk had no returns. NULLIF protects against division by zero.

Query 3 — Deliveries Per Route Per Day

Purpose: Count the deliveries made by each route on each delivery date.

```
SELECT
    r.route_code,
    r.route_name,
    d.delivery_date,
    COUNT(d.delivery_id) AS number_of_deliveries
FROM routes r
JOIN deliveries d
    ON r.route_id = d.route_id
GROUP BY
    r.route_id,
```

```
    r.route_code,
    r.route_name,
    d.delivery_date
ORDER BY
    d.delivery_date,
    r.route_code
```

Result:

route_code	route_name	delivery_date	number_of_deliveries
R001	Navi Mumbai North	2026-09-21	2
R002	Navi Mumbai South	2026-09-22	2
R003	Mumbai Central	2026-09-23	2
R004	Thane East	2026-09-24	2
R005	Thane West	2026-09-25	2
R006	Panvel Route	2026-09-26	2
R001	Navi Mumbai North	2026-09-28	2
R002	Navi Mumbai South	2026-09-29	2
R003	Mumbai Central	2026-09-30	2

Each route made two deliveries on its scheduled weekday. Routes R001, R002 and R003 appear twice because they have deliveries in both weeks of the sample data.

Query 4 — Unpaid Invoices

Purpose: List all invoices whose payment status is Unpaid, ordered by due date.

```
SELECT
    o.outlet_code,
    o.outlet_name,
    i.invoice_number,
    i.invoice_date,
    i.due_date,
    i.net_amount,
    i.payment_status
FROM invoices i
JOIN outlets o
    ON i.outlet_id = o.outlet_id
WHERE i.payment_status = 'Unpaid'
ORDER BY i.due_date
```

Result:

outlet_code	outlet_name	invoice_number	invoice_date	due_date	net_amount	payment_status
O002	Daily Needs Vashi	INV-1002	2026-09-21	2026-10-05	4800.00	Unpaid
O004	Sai Supermarket Nerul	INV-1004	2026-09-22	2026-10-06	7500.00	Unpaid
O008	Om Supermarket	INV-1008	2026-09-24	2026-10-08	6300.00	Unpaid
O010	Green Basket	INV-1010	2026-09-25	2026-10-09	5900.00	Unpaid
O012	New Panvel Mart	INV-1012	2026-09-26	2026-10-10	7100.00	Unpaid

Five invoices are unpaid, with a combined value of 31,600.00. One further invoice (INV-1006) is Partially Paid and is not included because the condition tests for exactly 'Unpaid'.

Query 5 — Outlets Returning More Than 20%

Purpose: Identify outlets whose total returns exceed 20% of their total deliveries. A CTE first totals delivered and returned units per outlet and the outer query filters on the calculated rate.

```
WITH outlet_totals AS (
    SELECT
        o.outlet_id,
        o.outlet_name,
        SUM(di.quantity_delivered) AS total_delivered,
        COALESCE(SUM(ri.quantity_returned), 0) AS total_returned
    FROM outlets o
    JOIN deliveries d
        ON o.outlet_id = d.outlet_id
    JOIN delivery_items di
        ON d.delivery_id = di.delivery_id
    LEFT JOIN returns r
        ON d.delivery_id = r.delivery_id
    LEFT JOIN return_items ri
        ON r.return_id = ri.return_id
        AND di.product_id = ri.product_id
    GROUP BY
        o.outlet_id,
        o.outlet_name
)
SELECT
    outlet_name,
    total_delivered,
    total_returned,
    ROUND(
        total_returned * 100.0
        / NULLIF(total_delivered, 0),
        2
    ) AS return_rate_percent
FROM outlet_totals
WHERE total_returned * 100.0
      / NULLIF(total_delivered, 0) > 20
ORDER BY return_rate_percent DESC
```

Result:

No rows returned.

The empty result is correct for this sample data: no outlet exceeds 20%. The table below shows the actual return rate of every outlet (computed with the same logic); the highest is Green Basket at 16.00%.

Outlet	Delivered	Returned	Return rate %
Green Basket	175	28	16.00
Sai Supermarket Nerul	350	42	12.00
Om Supermarket	195	22	11.28
Panvel Daily	225	25	11.11
New Panvel Mart	220	22	10.00
Thane Fresh	200	19	9.50
Royal Stores Sion	370	35	9.46
Westside Grocery	155	13	8.39
Daily Needs Vashi	325	20	6.15
City Mart Sion	250	13	5.20
Shree Grocery Nerul	275	13	4.73
Fresh Mart Vashi	330	15	4.55

The query logic is sound and will list outlets automatically if future data crosses the threshold. With the threshold lowered to 10%, four outlets would appear: Green Basket, Sai Supermarket Nerul, Om Supermarket and Panvel Daily.

14. SQL Concepts Demonstrated

Concept	Used In Project
SELECT	Yes
WHERE	Yes
ORDER BY	Yes
JOIN (INNER)	Yes
LEFT JOIN	Yes
GROUP BY	Yes
HAVING	Yes
COUNT	Yes
SUM	Yes
AVG / MIN / MAX	Yes
ROUND	Yes
COALESCE	Yes
NULLIF	Yes
CTE (WITH)	Yes
Subquery	Yes
UNION ALL	Yes
GRANT	Yes (roles.sql)
REVOKE	Yes (roles.sql)

15. Roles and Permissions

Role-based access control gives each type of user only the privileges the job needs. Two roles are defined. The statements below are stored in **roles.sql**, which is run after schema.sql.

bakery_manager — full privileges (all operations) on all tables and sequences.

bakery_clerk — limited operational privileges: can view all tables and insert deliveries, delivery items, returns and return items. The clerk cannot change products or prices, cannot touch invoices, and cannot update or delete delivery and return records (the REVOKE removes UPDATE and DELETE).

```
CREATE ROLE bakery_manager NOLOGIN;
CREATE ROLE bakery_clerk NOLOGIN;

GRANT USAGE ON SCHEMA public TO bakery_manager, bakery_clerk;

-- Manager: full privileges on all tables and sequences
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO bakery_manager;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO bakery_manager;

-- Clerk: read everything, work on the daily delivery/return tables
GRANT SELECT ON ALL TABLES IN SCHEMA public TO bakery_clerk;
GRANT INSERT, UPDATE, DELETE
    ON deliveries, delivery_items, returns, return_items
    TO bakery_clerk;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO bakery_clerk;

-- REVOKE: the clerk must not be able to delete delivery/return history
REVOKE UPDATE, DELETE
    ON deliveries, delivery_items, returns, return_items
    FROM bakery_clerk;
```

Role-based permissions are useful because they limit accidental or unauthorised changes (for example, a price edit or the deletion of delivery history), protect financial data, and make it clear who may do what.

16. Testing

Each constraint and permission was tested on PostgreSQL 16 against the seeded database using invalid or temporary records. Data-changing tests (CASCADE, SET NULL, DEFAULT) were rolled back so the sample data stays intact. The error messages in the table are the actual messages returned by PostgreSQL. Re-running the same statements in pgAdmin 4 should give the same messages.

Test	Expected result	Actual PostgreSQL output	Status
NOT NULL: NULL route_name rejected	Statement rejected with error	null value in column "route_name" of relation "routes" violates not-null constraint	Passed
UNIQUE: duplicate product_code rejected	Statement rejected with error	duplicate key value violates unique constraint "products_product_code_key"	Passed
CHECK: negative price rejected	Statement rejected with error	new row for relation "products" violates check constraint "chk_product_price"	Passed
CHECK: quantity 0 rejected	Statement rejected with error	new row for relation "delivery_items" violates check constraint "chk_delivery_quantity"	Passed
CHECK: invalid delivery_day rejected	Statement rejected with error	new row for relation "routes" violates check constraint "chk_route_day"	Passed
CHECK: due_date before invoice_date rejected	Statement rejected with error	new row for relation "invoices" violates check constraint "chk_due_date"	Passed
FOREIGN KEY: missing outlet rejected	Statement rejected with error	insert or update on table "deliveries" violates foreign key constraint "fk_delivery_outlet"	Passed
Composite PK: duplicate (delivery, product) rejected	Statement rejected with error	duplicate key value violates unique constraint "pk_delivery_items"	Passed
RESTRICT: delete route with outlets blocked	Statement rejected with error	update or delete on table "routes" violates foreign key constraint "fk_outlet_route" on table "outlets"	Passed
RESTRICT: delete product used in deliveries blocked	Statement rejected with error	update or delete on table "products" violates foreign key constraint "fk_delivery_item_product" on table "delivery_items"	Passed
CASCADE: delete delivery 1	Items and returns also deleted	delivery_items 3 to 0; returns 1 to 0; return_items of return 1 to 0	Passed
SET NULL: delete a route	Delivery kept, route_id becomes NULL	deliveries.route_id became NULL	Passed
DEFAULT values	Defaults applied when omitted	delivery_status=Delivered; payment_status=Unpaid; invoice_date=2026-10-01	Passed
Clerk: SELECT	Allowed	OK (count returned)	Passed
Clerk: INSERT delivery	Allowed	INSERT 0 1	Passed
Clerk: UPDATE products	Denied	permission denied for table products	Passed

Test	Expected result	Actual PostgreSQL output	Status
Clerk: DELETE deliveries	Denied	permission denied for table deliveries	Passed
Manager: UPDATE products	Allowed	UPDATE 1	Passed

17. Results and Discussion

The implemented database successfully stores bakery distribution information across eight related tables. Primary and foreign keys maintain relationships between entities, while constraints prevent invalid data. SQL queries allow the user to analyze product returns, deliveries, outlet supply, and unpaid invoices.

Key findings from the sample data: the bakery delivered 3,070 units in total across 18 deliveries, and 267 units were returned, leaving 2,803 units in net supply; White Bread is both the highest-volume product (1,155 units delivered) and the most-returned in absolute terms (118 units). No outlet exceeds the 20% return threshold; Green Basket is closest at 16.00%. Five of twelve invoices are unpaid, totalling 31,600.00.

All constraint and permission tests behaved as designed, and every required query ran successfully.

18. Advantages

- Centralized data storage
- Reduced duplication
- Better data integrity
- Easy searching
- Easier reporting
- Controlled user access
- Structured relationship management
- Easy future expansion

19. Limitations

- No graphical application interface.
- No real-time delivery tracking.
- No automated payment gateway.
- Sample data is static and small.
- No automatic inventory forecasting.
- Invoice amounts are stored values and are not computed from delivery and return quantities, so they can drift from the underlying data.
- Payments are tracked only by status; there is no payment table for partial amounts.

20. Future Scope

Planned extensions, in suggested order of development:

Step	Enhancement
1	Inventory management

Step	Enhancement
2	Delivery staff management
3	GPS delivery tracking
4	Online invoice generation
5	Payment tracking
6	Analytics dashboard
7	Mobile application

21. Conclusion

The Bakery Wholesale Distribution Route Database successfully demonstrates the design and implementation of a relational database using PostgreSQL. The system manages routes, outlets, products, deliveries, returns, and invoices while maintaining data integrity through keys and constraints. SQL queries provide useful information for analyzing distribution and return activities. The project also demonstrates role-based access control and database testing.

22. References

1. PostgreSQL Documentation, https://www.postgresql.org/docs/
2. PostgreSQL SQL Reference
3. DBMS course notes
4. pgAdmin 4 Documentation, https://www.pgadmin.org/docs/
5. diagrams.net Documentation